Week 6: Reviewing code

AI Development Course

Tested with Claude Code 2.1.296 on Ubuntu Linux (in a container) and the deal screener with Python 3.14.4 and 3.9.25, 12 October 2026, without signing in. Everything inside a signed-in session, on GitHub, or on a Mac or in Windows (WSL) is being tested on real Mac and Windows laptops. Nothing new to install: the same steps on every laptop.

Today

- Why several reviewers
- Read a diff on GitHub
- Make three reviewer agents, and check they can only read
- Run all three at once, into `REVIEW.md`
- Triage: `Fix` or `Won't fix`, with a reason
- Fix test first, read the diff yourself, merge
- The assignment, and next week

Why several reviewers

A publisher sends a manuscript to a fact checker, a lawyer and a copy editor. Each reads for **one** thing.

Reviewer	Its one question
`does-it-work`	Right answers? Strong enough tests?
`safe-and-private`	Passwords, keys or personal data where they shouldn't be?
`plain-english`	Could a non-coder understand, run and trust it?

Each one **only reports**: Must fix, Should fix or Nice to have, with file and line. **You** decide what to do.

Week 5's reviewers checked each step against the plan. This week: the whole build, for what the plan doesn't spell out.

The ready-made version (optional)

Anthropic's `pr-review-toolkit` plugin: six reviewers (comments, test gaps, silent failures, type design, code quality, simplifying).

```
/plugin install pr-review-toolkit@claude-plugins-official
/reload-plugins
/pr-review-toolkit:review-pr
```

- Its agent files have **no `tools` line**, so they **can** edit files
- `code-simplifier` is designed to rewrite code
- Built-in `/code-review`: its `--fix` option changes files

This week: **your own three reviewers, which can't change anything.**

Toolkit tested at commit f359d43b4060. Its pull request check may print a harmless error (being tested).

Read a diff

On your week 5 pull request, click **Files changed**.

- One box per file
- `+` lines (green): added. `-` lines (red): removed (being tested)
- Settings gear: **unified** (one column) or **split** (old beside new)
- `...` menu on a file: **View file** for the whole file (being tested)

Don't follow a part? Copy it, then in Claude Code:

```
Explain this change in plain English: <paste>
```

Deal screener's week 6 changes (works once week 6 is published): github.com/vasuchettyphd/ai-dev-course/compare/week-05...week-06

Branch first

In your terminal: `cd ~/semester-project && claude`, then `manual mode on`.

`Show me git status.` must say `nothing to commit, working tree clean`. Then:

```
Switch to main and pull the latest changes from GitHub. Then make a
new branch called review-fixes and switch to it.
```

Plain **Yes**. Check: "What branch am I on?" says `review-fixes`.

Week 5's rules carry over: plain Yes for git; No to anything that changes main, reset --hard, --force, git clean, rm -rf.

Copy the deal screener's reviewers

In a **second terminal window**:

```
cd ~ && git clone https://github.com/vasuchettyphd/ai-dev-course.git
cd ~/semester-project
mkdir -p .claude/agents
cp ~/ai-dev-course/example-project/.claude/agents/*.md .claude/agents/
ls .claude/agents
```

```
does-it-work.md  plain-english.md  safe-and-private.md
```

Second window: Mac Cmd + N; Windows, open Ubuntu again (not PowerShell). Already cloned, or ls shows nothing? cd ~/ai-dev-course && git pull, then copy again.

Restart Claude Code once: `/exit`, then `claude`. A new `agents` folder is only noticed at start. After that, edits are picked up in seconds.

A reviewer file: `does-it-work.md`

```
---
name: does-it-work
description: Use after a build to check that the code gives the right
  answers and has good tests. Only reports problems; never changes files.
tools: Read, Grep, Glob
---
You are a careful reviewer. Your one question is: does this code do what
the plan says, and give the right answers?

Report only. Never change any file.

First read PLAN.md, so you know what the project should do. ...

Look for:
- Numbers or results that do not match the formulas or rules in PLAN.md.
- Inputs that would break it or give a silly answer: zero, negative, ...

Write each finding like this:
- Label: Must fix, Should fix or Nice to have.
- Where: the file and line. ...
```

example-project/.claude/agents/ (description wrapped to fit the slide; in the file it's one line)

Make them fit your project

```
Read PLAN.md and the three reviewer files in .claude/agents/
(does-it-work.md, safe-and-private.md and plain-english.md). They were
written for another project, a rental deal screener. Adapt each one to
my project: keep its one question, the line "Report only. Never change
any file.", the three labels and the way each finding is written, and
change the "Look for" list so it fits what PLAN.md says my project does
and the data it uses. Keep the name and description lines as they are,
and keep the tools line exactly as it is: tools: Read, Grep, Glob. Then
show me what you changed in each file.
```

Edit prompts for the three files: plain **Yes** (not "don't ask again"). A prompt to change a `tools:` line: **No**. Read what changed.

Check: read-only

Every file must say exactly:

```
tools: Read, Grep, Glob
```

- `tools` is an **allowlist**: the reviewer can use only what's listed
- `Read` opens files, `Grep` searches in them, `Glob` finds them. None can change anything
- **No `tools` line** = every tool, including editing
- **No `Bash`**: a command can still change or delete files

Wrong? "Set the tools line in every file in .claude/agents/ to exactly: tools: Read, Grep, Glob. Change nothing else."

code.claude.com/docs/en/sub-agents

Check Claude can use them (being tested)

```
Which subagents can you use in this project?
```

Should name `does-it-work`, `safe-and-private`, `plain-english` (it may list built-in ones too).

Then check one can't edit:

```
Ask the does-it-work subagent to add the word "test" to the end of
README.md. Don't edit anything yourself and don't try another way.
Just tell me what the subagent said.
```

It should say it can't. But its instructions say never to edit, and the `tools:` line is what really stops it: a refusal alone proves nothing. An edit prompt? **No**, then check `tools:` again.

Optional: "Ask the does-it-work subagent which tools it has. Just tell me what it said." (Read, Grep, Glob; being tested)

Then: "Commit the three files in .claude/agents/ on this branch ..."

Run all three (being tested)

`/usage` first: write it down and keep it. Then one prompt:

```
Run the does-it-work, safe-and-private and plain-english reviewers in
parallel, as separate subagents, on this project. Then combine their
reports into REVIEW.md: one table with the columns #, Label, Finding
(file:line), Found by, Decision, Reason. If two or more reviewers found
the same problem, list it once and put every reviewer who found it in
"Found by". If they gave it different labels, keep the more serious
label and say so in the Finding. Sort the rows: Must fix first, then
Should fix, then Nice to have, and number them. Keep each finding's
wording close to what the reviewer wrote, and keep their file and line
numbers. Leave Decision and Reason empty. Do not add any finding the
reviewers did not report, and do not change any file except REVIEW.md.
After the table, list in a few words the areas where a reviewer said
"Nothing found".
```

While it runs

Three subagents read your files at once. Each report comes back in the chat or as a notification. **Wait for all three**, then Claude writes `REVIEW.md`.

Claude asks to	Choose
Create or edit `REVIEW.md`	**Yes**
Edit, create or delete **any other** file	**No**
`git status`, `git log`, `git diff`, `ls`	**Yes**
Change `main`, `--force`, `reset --hard`, `git clean`, `rm -rf`	**No**

Then read `REVIEW.md`, and commit it: "Review of the week 5 build".

Usage (being tested)

- Every subagent sends its own requests, which "count toward the same usage limits as your main conversation"
- So three reviewers use **more** than one conversation. We have no published number for how much more
- `/usage` again: compare with before. It shows the **subagents' share** (`d` / `w`)
- Its `Prompt cache (main)` line counts the main conversation only, not subagents
- At a limit: wait for the reset, as in week 5

code.claude.com/docs/en/sub-agents and code.claude.com/docs/en/costs

Triage: you decide (being tested)

Every finding gets a **Decision** (`Fix` or `Won't fix`) and a **Reason**.

Check each finding first. Reviewers can be wrong. The deal screener had three wrong line numbers (#5, #11, #13) and one partly wrong finding (#10).

```
Check finding #1 in REVIEW.md against the code. Is it right, and are
the file and line numbers right? Explain in plain English. Don't
change any file.
```

- Must fix: `Fix`, unless the finding is wrong (then give the evidence)
- Every `Won't fix` needs a real reason: say **why**
- Don't fix everything blindly; don't wave everything away

The deal screener's `REVIEW.md`: three rows

13 findings: 1 Must fix, 5 Should fix, 7 Nice to have. 10 `Fix`, 3 `Won't fix`.

#	Label	Finding	Decision	Reason
1	Must fix	README and a test say "listing B from the plan", but PLAN.md has no listings A or B	Fix	Really a wording fix: they now point at the task plan, which has the listings
8	Nice to have	A tiny negative number prints "-$0.00"	Fix	Real bug: numbers are now rounded before formatting, with tests written first
10	Nice to have	No .gitignore in the project	Won't fix	Partly wrong: the course repo's own .gitignore already covers it

Shortened. Full file: example-project/REVIEW.md. Commit your triage: "Triage the review".

Fix: one at a time, test first

Must fix first. A finding that changes what the program **does**:

```
Fix finding #1 in REVIEW.md, test first: write a test that fails
because of this problem, run the tests and show me it fails, then
change the code until all the tests pass. Change nothing else. Then
commit on this branch with the message "Fix review #1: <a few words>".
```

A **wording** finding: "Fix finding #5 in REVIEW.md. It is a wording change, so no new test: make the change, run all the tests, and commit ..."

Deal screener #8: new tests failed first (`ImportError: cannot import name 'percent'`), then 37 tests passed.

Check, push, read, merge

1. Second terminal window: run your tests. **Screenshot.** Run your program on your week 5 hand-worked example: same answer?
2. In Claude Code:

   ```
   Push the review-fixes branch to GitHub and open a pull request into
   main titled "Review fixes".
   ```

3. **Files changed:** `REVIEW.md`, the three agent files, and the files your fixes changed (code, tests, and any README or PLAN.md you reworded): nothing else. Read each fix's green and red lines **yourself**
4. **Merge pull request**, **Confirm merge** (or **Squash and merge**; being tested), then "Switch back to main and pull the latest changes from GitHub."

Deal screener after its fixes: Ran 39 tests, OK. Committed to main? Week 5's rescue steps.

Assignment

- The **merged fix PR** address: three reviewer files, `REVIEW.md`, the fixes
- Screenshot: **tests passing** after the fixes
- Reflection: the **most useful reviewer**, what you did about it (quote a green or red line from its fix, say what it does); one finding you **disagreed** with, and why

Criterion	Points
Three reviewer files, read-only, each with a clear focus	2
`REVIEW.md`: every finding triaged, every `Won't fix` reasoned	3
Merged fix PR: Must fixes done, tests passing	3
Reflection	2

Next week: Running what you build

Bring:

- your merged review fixes
- your `REVIEW.md`, and the finding you disagreed with
- your `/usage` numbers from this week